

Lecture-10. Pygame II.

1. What Are Sprites?

◆ Definition

A **sprite** is a 2D image or object used in a game.

◆ Why use sprites?

- Organize game objects (player, enemies, bullets)
- Built-in support for:
 - Drawing
 - Updating
 - Grouping

◆ Basic Sprite Example

```
import pygame

class Player(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((50, 50))
        self.image.fill((0, 128, 255))
        self.rect = self.image.get_rect()
        self.rect.x = 100
        self.rect.y = 100

    def update(self):
        self.rect.x += 1
```

◆ Sprite Groups

```
all_sprites = pygame.sprite.Group()

player = Player()
```

```
all_sprites.add(player)

all_sprites.update()
all_sprites.draw(screen)
```

✓ Groups allow updating and drawing multiple objects at once.

💣 2. Collision Detection

♦ What is Collision?

When two objects touch or overlap.

♦ Basic Collision Check

```
if player.rect.colliderect(enemy.rect):
    print("Collision!")
```

♦ Using Sprite Groups

```
hits = pygame.sprite.spritecollide(player, enemies, False)

if hits:
    print("Player hit!")
```

♦ Advanced Collision (Pixel Perfect)

```
pygame.sprite.collide_mask(sprite1, sprite2)
```

📌 Requires masks:

```
self.mask = pygame.mask.from_surface(self.image)
```

3. Movement Logic

◆ Basic Movement with Keyboard

```
keys = pygame.key.get_pressed()

if keys[pygame.K_LEFT]:
    player.rect.x -= 5
if keys[pygame.K_RIGHT]:
    player.rect.x += 5
```

◆ Smooth Movement (Velocity)

```
self.velocity = 5

def update(self):
    keys = pygame.key.get_pressed()

    if keys[pygame.K_LEFT]:
        self.rect.x -= self.velocity
```

◆ Diagonal Movement Fix

Without fix → faster diagonal movement ❌

Fix using normalization:

```
import math

dx, dy = 0, 0

if keys[pygame.K_LEFT]:
    dx = -1
if keys[pygame.K_RIGHT]:
    dx = 1
if keys[pygame.K_UP]:
    dy = -1
if keys[pygame.K_DOWN]:
    dy = 1

length = math.hypot(dx, dy)
if length != 0:
```

```
dx /= length
dy /= length

self.rect.x += dx * speed
self.rect.y += dy * speed
```

◆ Boundaries (Keep Player on Screen)

```
if self.rect.left < 0:
    self.rect.left = 0
if self.rect.right > WIDTH:
    self.rect.right = WIDTH
```



4. Combining Everything

◆ Game Loop Example

```
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    all_sprites.update()

    hits = pygame.sprite.spritecollide(player, enemies, False)
    if hits:
        print("Game Over")

    screen.fill((0, 0, 0))
    all_sprites.draw(screen)
    pygame.display.flip()
```